

Chef's Mind AI – Project Context & Manifest

This document provides a complete overview of the Chef's Mind AI project, including its purpose, architecture, completed milestones and the ongoing UI redesign. It replaces older context guides and should be used by KiloCode and other LLM agents as the primary source of project context.

Purpose

Chef's Mind AI is an intelligent multi-agent assistant designed for professional kitchens. It helps chefs and managers create and refine recipes, calculate costs and budgets, research culinary information and generate media assets such as images and videos. The system uses multiple large-language model agents orchestrated by a central router to route each request to the most appropriate agent.

Architecture & Modules

The project consists of several layers:

- **Frontend** – A Next.js application (currently migrated to Next 14) using Tailwind CSS and shadcn/ui components. The frontend provides the chat interface, sidebars and controls. A new UI is being developed to mirror the ChatGPT/Projects experience: a left sidebar for agent selection, a central chat, and a right sidebar for search, library and calendar. Input bar includes attachments, model selection, microphone and send.
- **Backend** – A Node.js/Express server exposing endpoints (e.g., `/api/enhanced-agent/chat`) that proxy requests to the orchestrator. It uses Drizzle ORM and Postgres for data persistence. The backend enforces RBAC, OAuth and safe-mode confirmations.
- **Agents** – Specialized LLM agents for different domains:
 - **Accountant** – manages budgets, orders, suppliers and cost calculations.
 - **Su-Chef** – generates and edits recipes, technical cards and kitchen workflows.
 - **Researcher** – uses Perplexity or other search APIs to find information.
 - **Media Studio** – generates images and videos via external AI providers (Gemini, GPT-4o, etc.) and uses a prompt enhancer to adapt prompts to each provider.
 - **QA/Gate** – optional agent that critiques or verifies outputs for quality assurance.
- **Orchestrator (Agent Router)** – A routing layer that classifies user intent and forwards requests to the appropriate agent. It also supports caching of recent intents and provides a supervisor agent for complex multi-step workflows. The project currently uses a simple classifier based on keywords but is migrating to a TypeScript implementation inspired by the Agent Squad examples.
- **MCP Services** – Microservices that perform specific actions on behalf of the agents, such as a prompt-enhancer for media generation or accounting operations. These services may run separately from the main server.
- **Data Persistence** – A set of Drizzle migrations and APIs that manage orders, suppliers and chat history. The data layer includes backup, restore and calendar integration.

Completed Blocks (Milestones)

- **Block 1 – Project Foundation:** Set up repository, created baseline chat UI, configured environment (Docker, Drizzle, RBAC). Added essential endpoints and baseline tests.
- **Block 2 – Multi-Agent Routing:** Implemented a new orchestrator (`server/agents/orchestrator.ts`) that classifies intents into five categories (cooking, finance, research, media and QA). Added a supervisor agent for complex queries and a lightweight caching mechanism. Updated `/api/enhanced-agent/chat` to use the orchestrator. Created unit and e2e tests. Added documentation `docs/AGENT_ROUTING_DESIGN.md`.
- **Block 3 – Data Persistence:** Added CRUD endpoints for orders and suppliers. Implemented chat history storage and retrieval. Added calendar integration for events. Created backup operations and updated `docs/DATA_LAYER_OVERVIEW.md`. Added UI pages for orders and suppliers with full CRUD logic.
- **Block 4 – UI Redesign (in progress):** Began replacing the old chat interface with a ChatGPT-like workspace:
 - Removed the “main chat” button – the main chat is now the root page. The agent selector now lists only Accountant, Su-Chef, Researcher and Media Studio.
 - Designed left sidebar to list agents and a Settings button. Right sidebar includes search, library and calendar (and optionally a database view).
 - Added bottom input bar with attachments, model selector, mic and send. The bar also toggles between auto and specific models (GPT-4o, Gemini, etc.).
 - Selected the **Agent Squad-style** orchestrator (option #4) as the basis for multi-agent routing.
 - Integrated external references from lobe-chat and open-webui to guide implementation (modern design AI agent workspace and knowledge base support ¹).

Next Steps

1. **Complete Block 4 (UI Redesign)** – Finish implementing the new UI in the `frontend-enhanced` project using patterns from lobe-chat and open-webui. Ensure agent switching, sidebars and input bar work seamlessly.
2. **Integrate Prompt Enhancer** – Create a TypeScript module for prompt enhancement (e.g., `/server/media/promptEnhancer.ts`) that normalizes and adapts prompts for each media model. Attach it as a tool for the Media Studio agent.
3. **Upgrade Orchestrator** – Incorporate patterns from `external/agent-squad/typescript` into `server/agents/orchestrator.ts` to support a better classifier and supervisor agent.
4. **Refactor MCP Services** – Evaluate whether MCP servers (e.g., Playwright, ComfyUI) are needed. Keep ComfyUI and Flowise repositories separate; do not import them into the core UI.
5. **Update Context Files** – Replace any older context files (e.g., `MASTER_CONTEXT_GUIDE_FULL.md`) with this document and the integration guide. Update the KiloCode memory bank to reference `PROJECT_CONTEXT.md` and `EXTERNAL_SOURCES_UI.md`.

How to Use This Manifest

1. Treat this file as the single source of truth for project structure and goals. Whenever you add a new feature or modify architecture, update this document.
2. Read `docs/EXTERNAL_SOURCES_UI.md` for details on the external repositories and `docs/INTEGRATION_GUIDE.md` for instructions on integrating their components.
3. KiloCode and other LLM assistants should load this context at the start of a session to understand the current state of Chef's Mind AI and avoid outdated instructions.

4. When generating or modifying code, adhere to the guidelines in this manifest and integration guide.

1 GitHub - lobehub/lobe-chat: LobeHub - an open-source, modern design AI Agent Workspace. Supports multiple AI providers, Knowledge Base (file upload / RAG), one click install MCP Marketplace and Artifacts / Thinking. One-click FREE deployment of your private AI Agent application.
<https://github.com/lobehub/lobe-chat>